



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN



LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO
COMPUTADORA

EJERCICIOS DE CLASE N° 05

NOMBRE COMPLETO: Velazquez Caudillo Osbaldo

N° de Cuenta: 320341704

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 06

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 24/09/2025

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. **Actividades realizadas.** Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa

Ejercicio

1.- Importar por separado y agregar jerarquía:

-Cuerpo con cabeza y cola

-mandíbula inferior

- Cada una de las 4 patas independiente

2.- Agregar rotaciones a las patas de forma independiente para que se muevan como si fuera a avanzar Goddard, limitar la rotación a 45° en Cada sentido

Una vez que exportamos nuestros .obj de blender, lo que hacemos es importarlos para poder hacer uso de ellos.

```
Camera camera;
Model Goddard_C;
Model Goddard_M;
//PDD=pata_derecha_delantera
Model Goddard_PDD;
//PID=pata_izquierda_delantera
Model Goddard_PID;
//PDT
Model Goddard_PDT;

Model Goddard_PIT;

Skybox skybox;
```

```
//cuerpo
Goddard_C = Model();
Goddard_C.LoadModel("Models/cuerpo_goddarobj.obj");
//mandibula
Goddard_M = Model();
Goddard_M.LoadModel("Models/mandibulaobj.obj");

Goddard_PDD = Model();
Goddard_PDD.LoadModel("Models/pata_derecha_delanteraobj.obj");

Goddard_PID = Model();
Goddard_PID.LoadModel("Models/pata_izquierda_delanteraobj.obj");

Goddard_PDT = Model();
Goddard_PDT.LoadModel("Models/pata_derecha_traseraobj.obj");

Goddard_PIT = Model();
Goddard_PIT.LoadModel("Models/pata_izquierda_traseraobj.obj");
```

Posterior a ello lo que hacemos es con ayuda de la jerarquía acomodar las partes del cuerpo que se pidieron como lo fue, la mandíbula y las 4 patas,

```
//-----*INICIA DIBUJO DE NUESTROS DEMOS OBJETOS-----*
//Goddard
color = glm::vec3(0.0f, 0.0f, 0.0f); //modelo de goddard de color negro

model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.46f, -1.5f));
modelaux = model;
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Goddard_C.RenderModel(); //modificar por el modelo sin las 4 patas y sin cola
//color = glm::vec3(0.0f, 0.0f, 0.0f);
//En sesión se separara una parte del modelo de Goddard y se unirá por jerarquía al cuerpo
//modelaux = model;
//Cola
//Mandibula
model = modelaux;
color = glm::vec3(1.0f, 0.0f, 0.0f);
model = glm::translate(model, glm::vec3(-2.38f, 1.1f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Goddard_M.RenderModel();

//Siguientes modelos
/* Ejercicio:
1.- Separar las 4 patas de Goddard del modelo del cuerpo, unir por medio de jerarquía cada pata al cuerpo de Goddard
2.- Hacer que al presionar una tecla cada pata pueda rotar un máximo de 45° "hacia adelante y hacia atrás"
*/

//pata delantera derecha
model = modelaux;
color = glm::vec3(0.78f, 0.80f, 0.85f);
model = glm::translate(model, glm::vec3(-1.2f, -0.3f, 0.7f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Goddard_PDD.RenderModel();

// pata delantera izquierda
model = modelaux;
color = glm::vec3(0.78f, 0.80f, 0.85f);
model = glm::translate(model, glm::vec3(-1.2f, -0.24f, -0.5f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Goddard_PID.RenderModel();

//pata trasera derecha
model = modelaux;
color = glm::vec3(0.78f, 0.80f, 0.85f);
model = glm::translate(model, glm::vec3(1.0f, -0.9f, 0.7f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Goddard_PDT.RenderModel();

// pata delantera izquierda
model = modelaux;
color = glm::vec3(0.78f, 0.80f, 0.85f);
model = glm::translate(model, glm::vec3(-1.2f, -0.24f, -0.5f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Goddard_PID.RenderModel();

//pata trasera izquierda
model = modelaux;
color = glm::vec3(0.78f, 0.80f, 0.85f);
model = glm::translate(model, glm::vec3(1.0f, -0.9f, 0.7f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Goddard_PDT.RenderModel();

//pata trasera derecha
model = modelaux;
color = glm::vec3(0.78f, 0.80f, 0.85f);
model = glm::translate(model, glm::vec3(1.0f, -0.9f, 0.7f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Goddard_PDT.RenderModel();

//pata trasera izquierda
model = modelaux;
color = glm::vec3(0.78f, 0.80f, 0.85f);
model = glm::translate(model, glm::vec3(0.8f, -1.05f, -0.5f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Goddard_PIT.RenderModel();
```

Le agregamos las traslaciones necesarias para que quede acorde al modelo original de Goddard, cada una de las patas tiene una articulación para que se puedan mover de manera independiente.

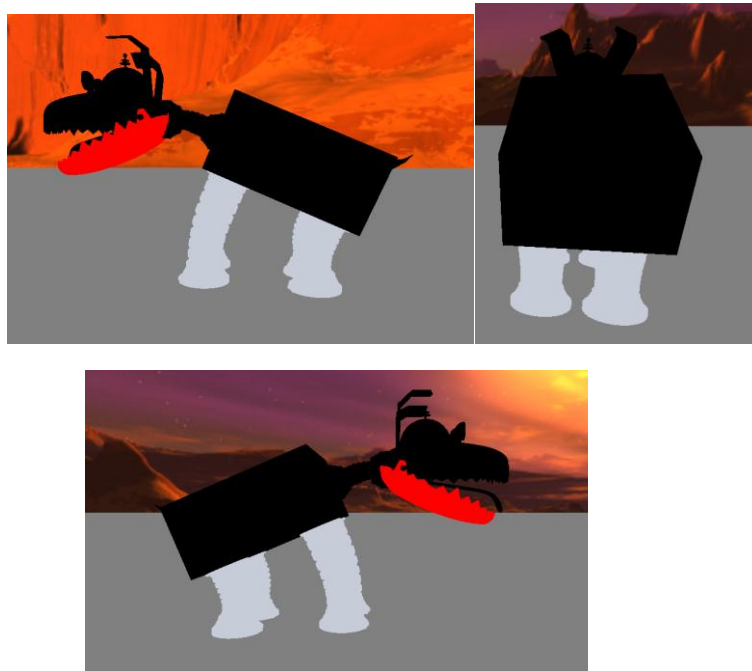
Después pasamos a las rotaciones de 45 grados en las patas, para lo cual en el window.cpp se agrego lo siguiente:

```
f (key == GLFW_KEY_G)

    static int dir = 1;
    theWindow->articulacion2 += 5.0f * dir;
    if (theWindow->articulacion2 >= 45.0f) { theWindow->articulacion2 = 45.0f; dir = -1; }
    else if (theWindow->articulacion2 <= -45.0f) { theWindow->articulacion2 = -45.0f; dir = 1; }
```

Para cada una de las teclas que controlan el movimiento de la articulación la variable dir (dirección) lo que hace es que guarda a dirección actual si es 1 se mueve adelante si es -1 es hacia atrás y lo que hace es sumar de 5 en 5 o -5 dependiendo del caso con las condiciones nos aseguramos de que no pueda sobrepasar de 45 grados y -45 respectivamente con esto, lo que logramos es que simule el movimiento de las patas de Goddard

Ejecución



2. Problemas presentados. Listar si surgieron problemas a la hora de ejecutar el código

Cuando estaba exportando los .obj de blender si me encontré con varios problemas de que no se seleccionaba todo, sobre todo con el cuerpo la cabeza y la cola que se movían o parecían que estaban separados.

3. Conclusión:

No fue difícil gracias al video que posterior al día de la práctica revise, sin embargo, en el laboratorio la verdad es que no me fue de mucha ayuda, porque iba muy rápido y me perdí al inicio, entonces eso no me ayudo mucho, pero como mencione anteriormente los videos fueron de mucha ayuda.

En general el ejercicio no me pareció malo, pero la explicación si estuvo muy deficiente.